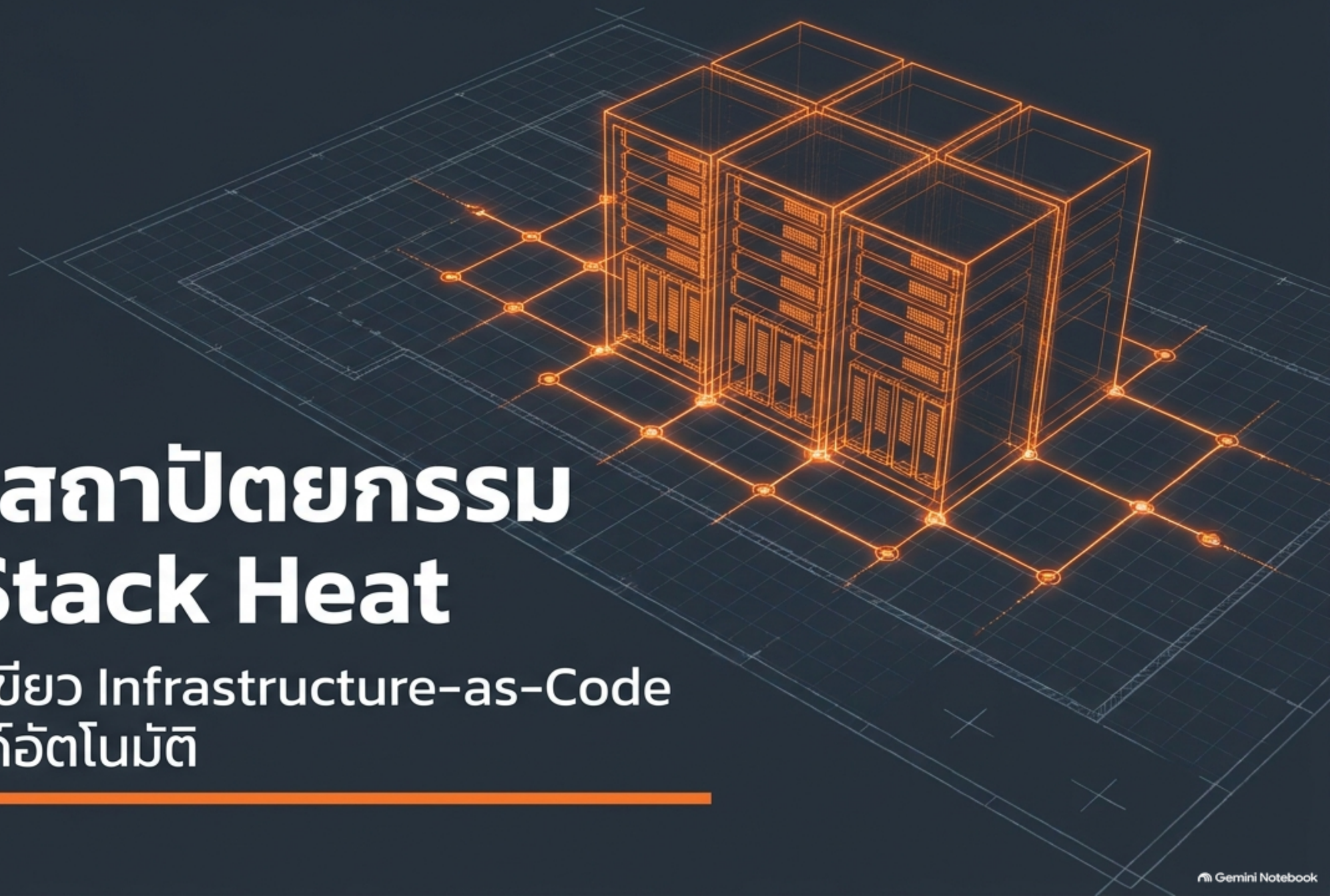


เจาะลึกสถาปัตยกรรม OpenStack Heat

เปลี่ยนพิมพ์เขียว Infrastructure-as-Code สู่ระบบคลาวด์อัตโนมัติ

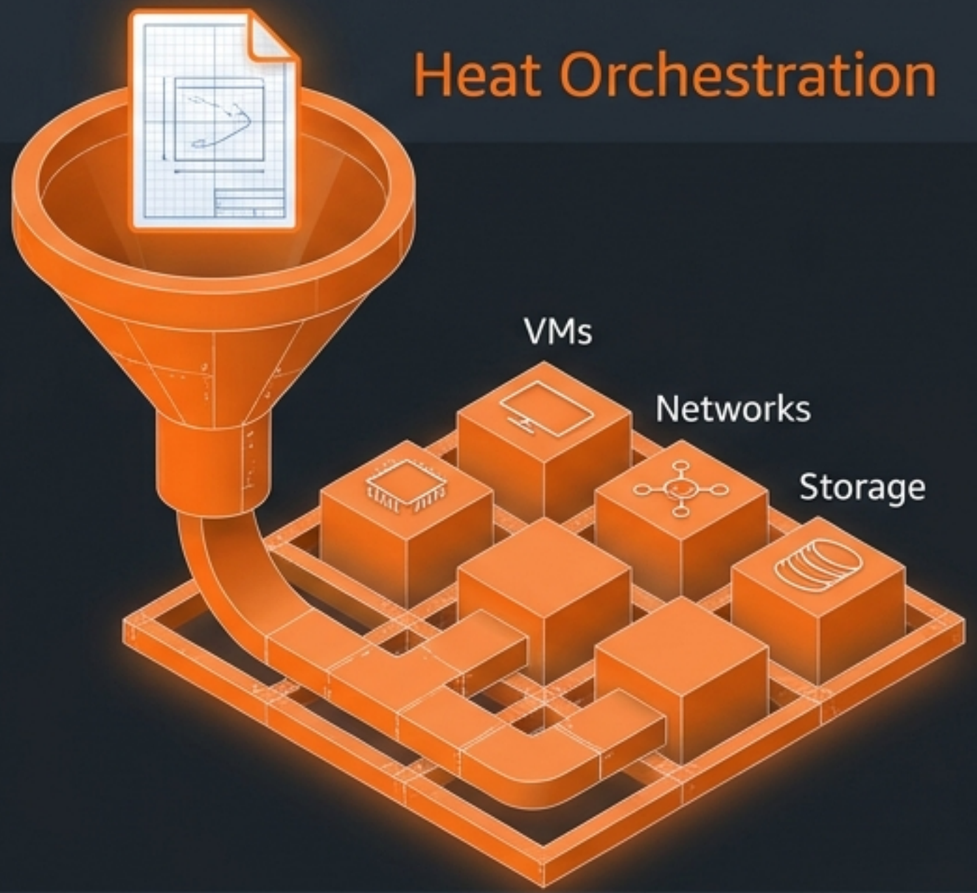
Gemini Notebook

ยกระดับการจัดการคลาวด์สู่ Infrastructure-as-Code

Manual Provisioning



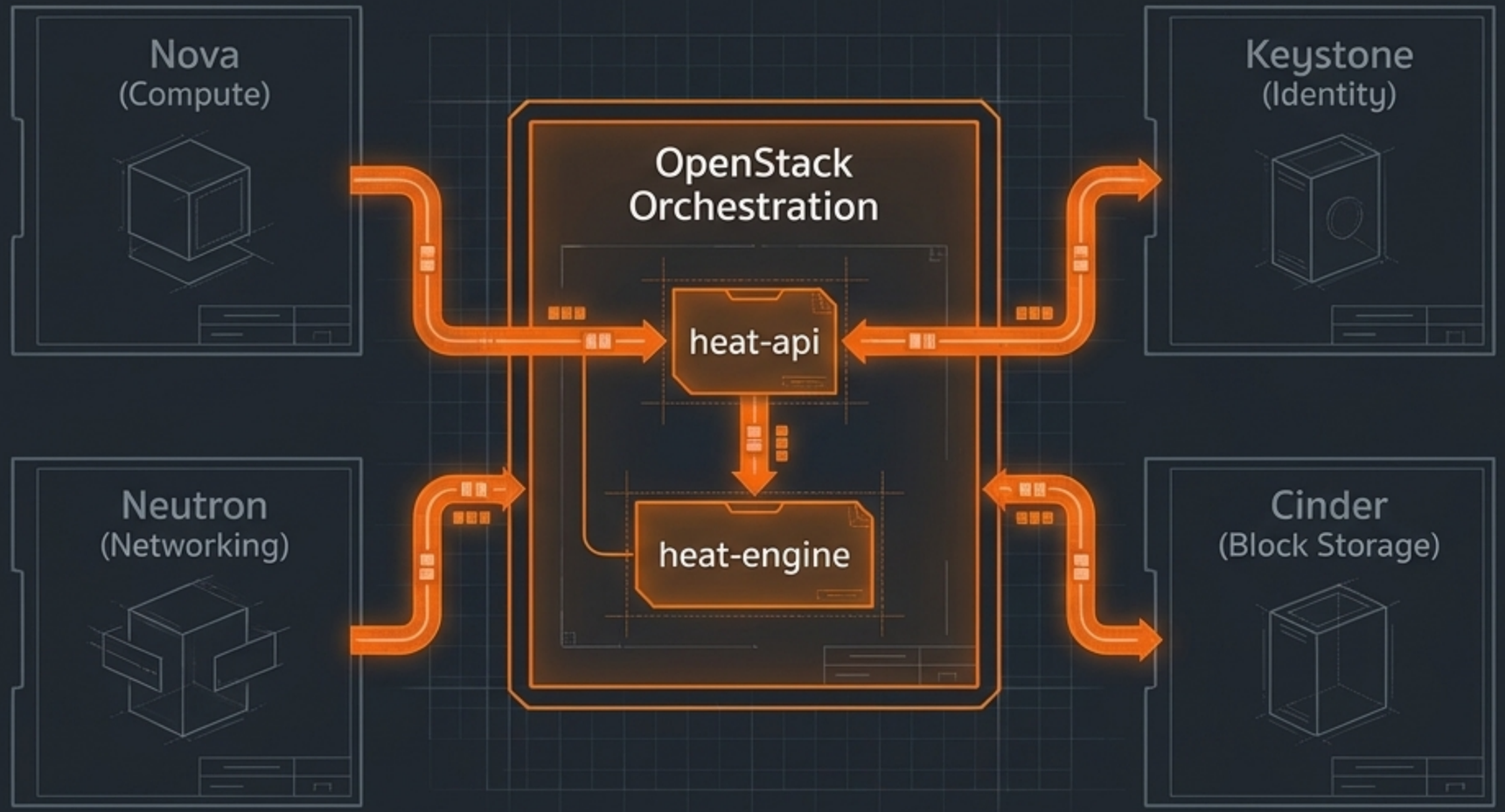
Heat Orchestration



- เลิกการตั้งค่าแบบ Manual ที่ซ้ำซ้อน และเสี่ยงต่อข้อผิดพลาด
- ใช้โค้ด (Template) เป็นศูนย์กลางในการประกาศสถานะของระบบ (Declarative)
- Heat เป็นเครื่องมือสร้างและจัดการวงจรชีวิตทรัพยากรทั้งหมดใน OpenStack โดยอัตโนมัติ

Gemini Notebook

ศูนย์กลางระบบประสาทของ OpenStack



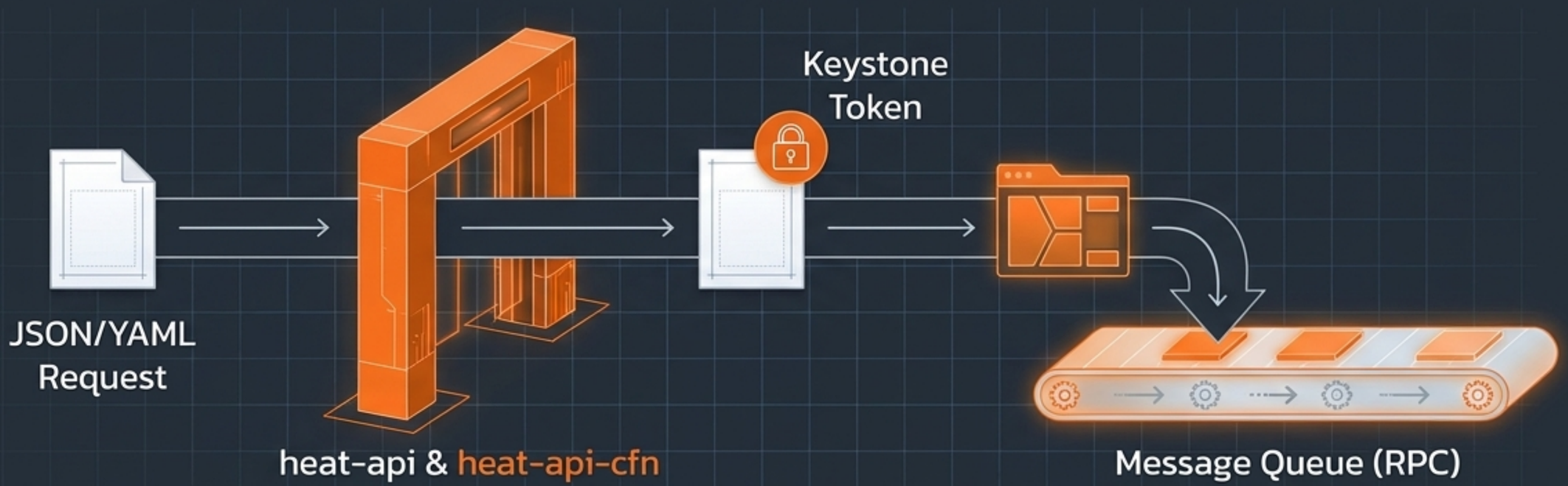
Receiver:
รับคำสั่งคา รับคำสั่งผ่าน HTTP(S) REST API

Authenticator:
ยืนยันสิทธิ์ผ่าน Keystone

Orchestrator:
สั่งการ Nova, Neutron, Cinder ฯลฯ ผ่าน API ของแต่ละบริการแบบขนาน

Gemini Notebook

Heat API: ประตูรับคำสั่งและตรวจสอบสิทธิ์



API

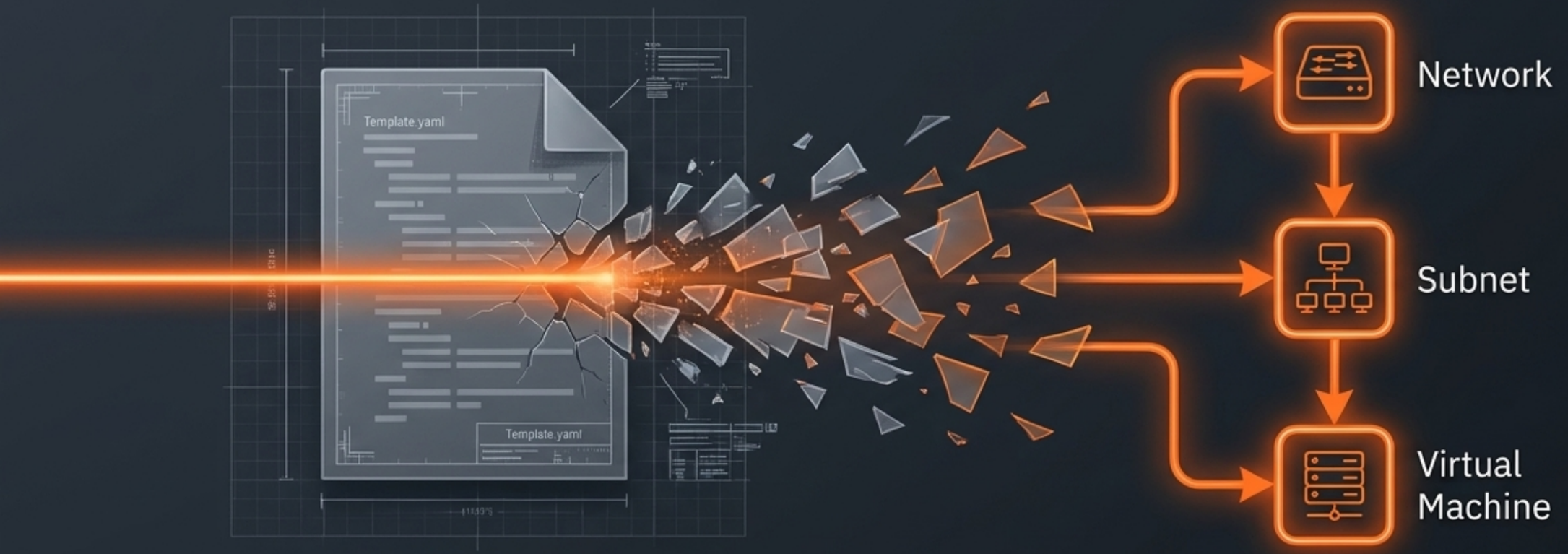
รองรับทั้ง OpenStack-native REST API และ CloudFormation-compatible API (heat-api-cfn)

ตรวจสอบ Token และสิทธิ์ Role-Based Access Control (RBAC) ผ่าน Keystone

แปลงคำสั่ง (Translate) และส่งเข้าสู่ระบบ Message Queue เพื่อลดคอขวด (Asynchronous)

Gemini Notebook

Heat Engine: สมองกลวิเคราะห์และเรียงลำดับการสร้าง



1. Parse & Validate: ดึงข้อความจาก Queue มาตีความโครงสร้าง YAML

2. Dependency Resolution: แปลงทรัพยากรเป็นกราฟทิศทาง (Directed Acyclic Graph - DAG) เพื่อหาลำดับก่อน-หลัง (เช่น สร้าง Network ก่อน VM เสมอ)

3. Execute Calls: ส่งคำสั่งไปยัง OpenStack Service APIs (Nova, Neutron) ตามลำดับที่คำนวณไว้

Gemini Notebook

ชำแหละโครงสร้าง Heat Orchestration Template (HOT)

The Blueprint

Diagnostic Panel

The Reality

```yaml
parameters:
  flavor_size:
    type: string
    description: VM Flavor
  image_name:
    type: string
    description: OS Image

resources:
  my_server:
    type: OS::Nova::Server
    properties:
      flavor: { get_param: flavor_size }
      image: { get_param: image_name }

outputs:
  server_ip:
    description: Public IP of the server
    value: { get_attr: [my_server, first_address] }
```



Parameters:
ตัวแปรที่รับค่าจากผู้ใช้ตอนรัน
(เช่น ขนาด Flavor, ชื่อ Image)



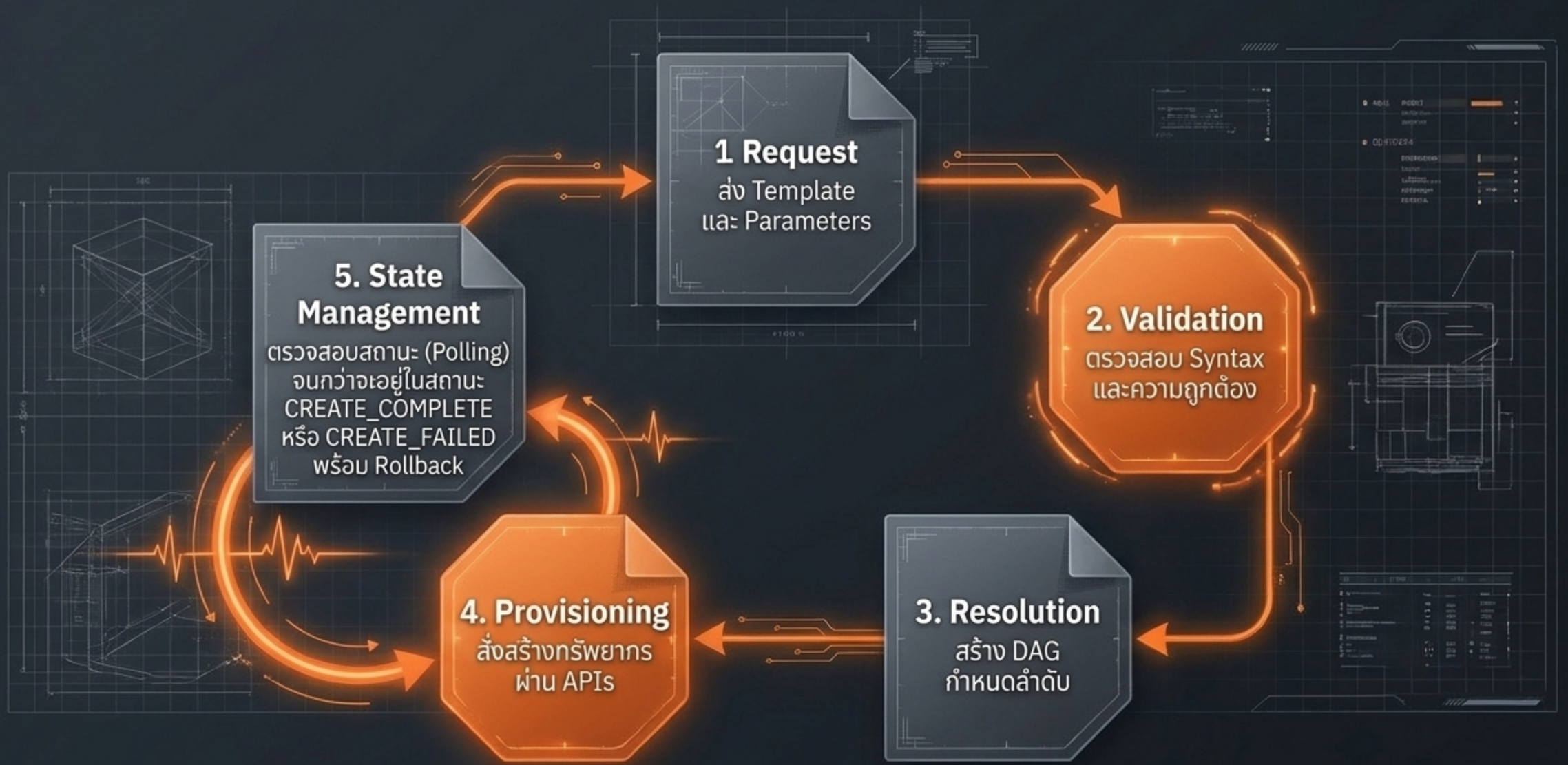
Resources:
หัวใจหลักที่กำหนดว่าต้องการสร้างอะไร
(VM, Volume, Security Group)



Outputs:
ข้อมูลที่ส่งกลับมาหลังสร้างเสร็จ
(เช่น Public IP สำหรับใช้งานต่อ)

Gemini Notebook

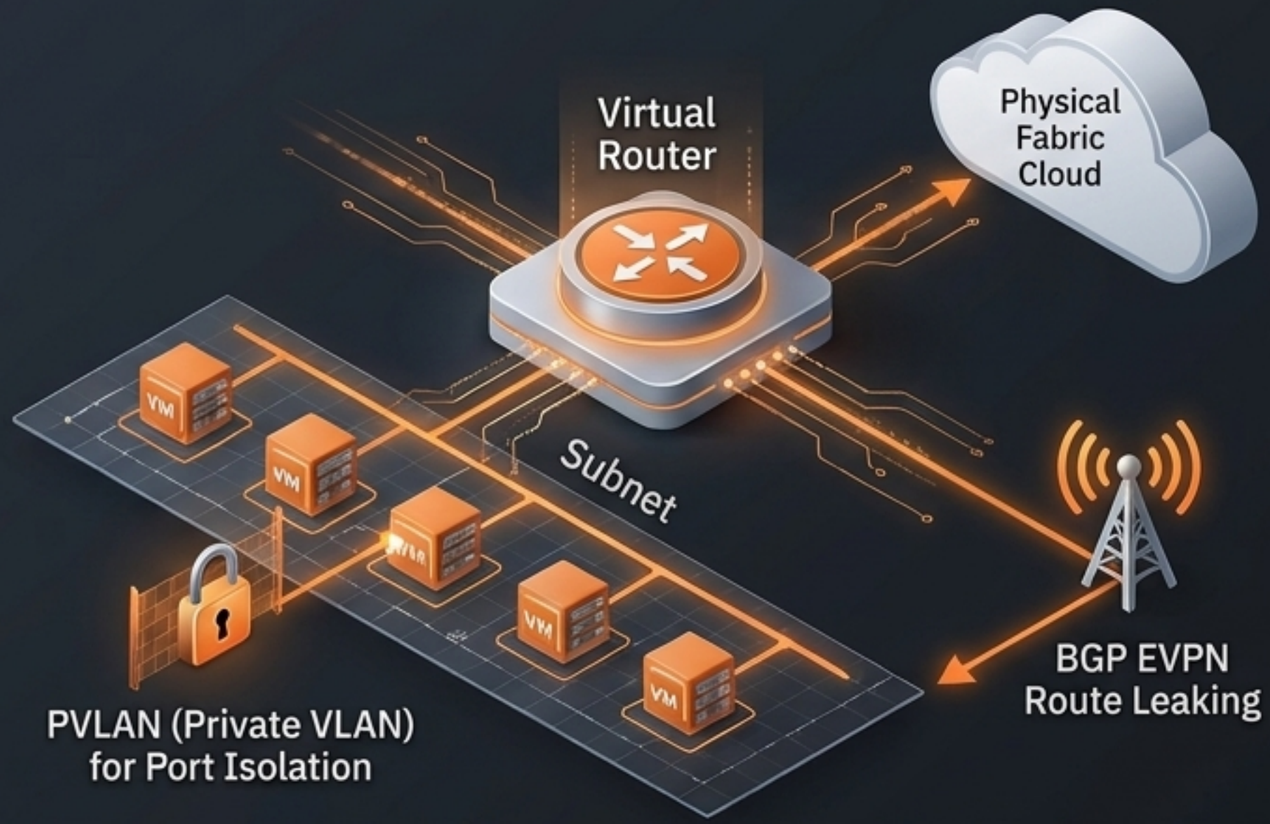
วงจรชีวิตของการสร้าง Stack อัตโนมัติ (Orchestration Lifecycle)



Gemini Notebook

สร้างและจัดการเครือข่ายซับซ้อนในพริบตา (Neutron Integration)

```yaml
resources:
  my_router:
    type: OS::Neutron::Router
    properties:
      name: my-virtual-router
      external_gateway_info:
        network: public_network
  my_network:
    type: OS::Neutron::Net
    properties:
      name: my-internal-network
  my_subnet:
    type: OS::Neutron::Subnet
    properties:
      network: { get_resource: my_network }
      cidr: 192.168.10.0/24
      gateway_ip: 192.168.10.1
  router_interface:
    type: OS::Neutron::RouterInterface
    properties:
      router: { get_resource: my_router }
      subnet: { get_resource: my_subnet }
```



สั่งการ Neutron เพื่อประกอบ Router, Network, และ Subnet อัตโนมัติ

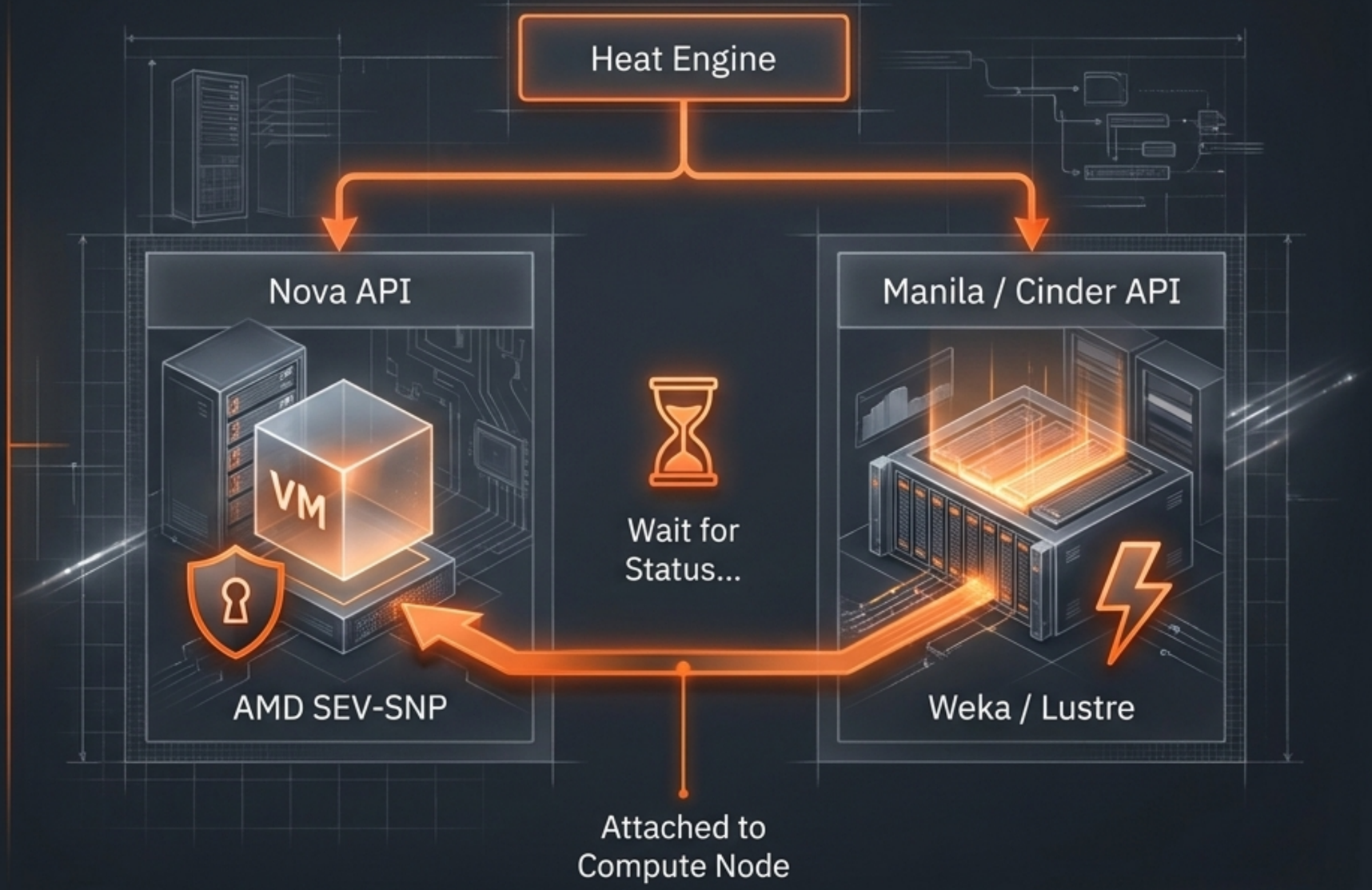
รองรับฟีเจอร์เครือข่ายระดับลึก เช่น Private VLAN (PVLAN) สำหรับ Port Isolation และ BGP EVPN route leaking สู่ Physical Fabric (ฟีเจอร์ล่าสุดจาก 2026.2)

กำหนด Security Groups และผูก IP อัตโนมัติโดยไม่มีข้อผิดพลาดจากมนุษย์

Gemini Notebook

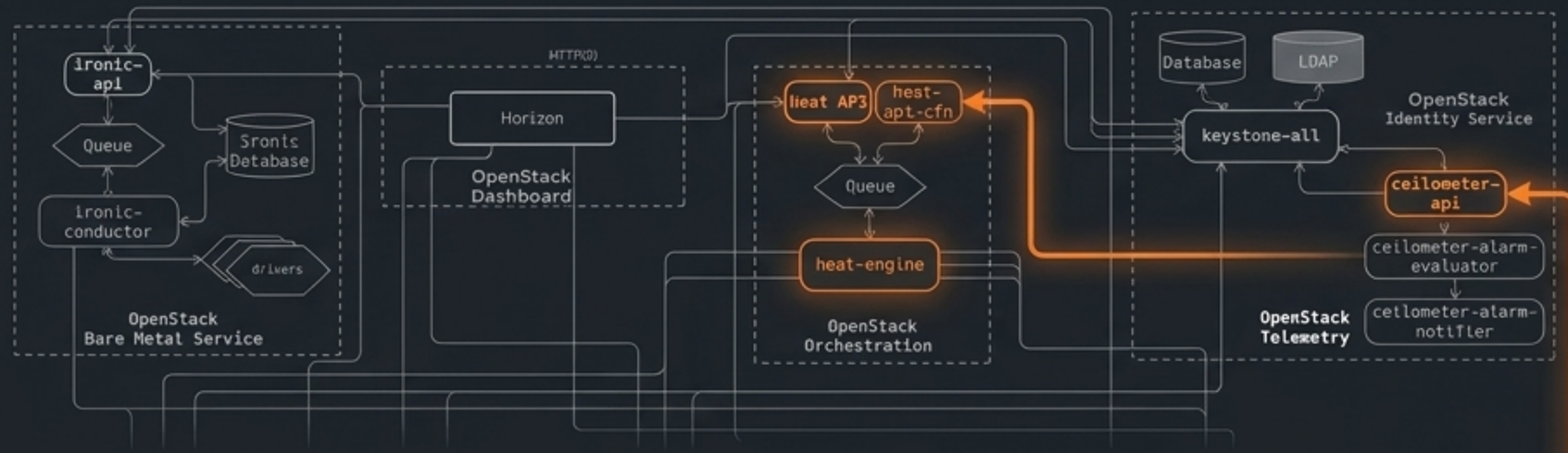
จัดสรรประมวลผลและพื้นที่จัดเก็บระดับสูง (Nova, Cinder, Manila)

- **ประสานการทำงานข้ามบริการ:** รอให้ Storage สร้างเสร็จก่อนทำการ Attach เข้ากับ Compute Node
- **รองรับ** Workload **ขั้นสูงผ่าน** Template: สั่งเปิดใช้งาน AMD SEV-SNP (Confidential Computing) หรือจัดสรร vGPU ผ่าน Cyborg
- **ผูก** High-Performance Storage (เช่น Lustre/Weka ผ่าน Manila) สำหรับงาน AI/HPC ได้ทันที



Gemini Notebook

การขยายระบบอัตโนมัติ (Auto-Scaling & Telemetry)





Webhook



Monitor: Ceilometer
เก็บข้อมูล Metrics (เช่น CPU Load)

Alarm: Aodh
แจ้งเตือนเมื่อโหลดเกินกำหนด

Action:
ทริกเกอร์ Webhook กลับมาที่ Heat API

Scale: Heat Engine
เพิ่มจำนวน Instance ตามที่ตั้งค่าไว้ใน OS::Heat::AutoScalingGroup โดยอัตโนมัติ

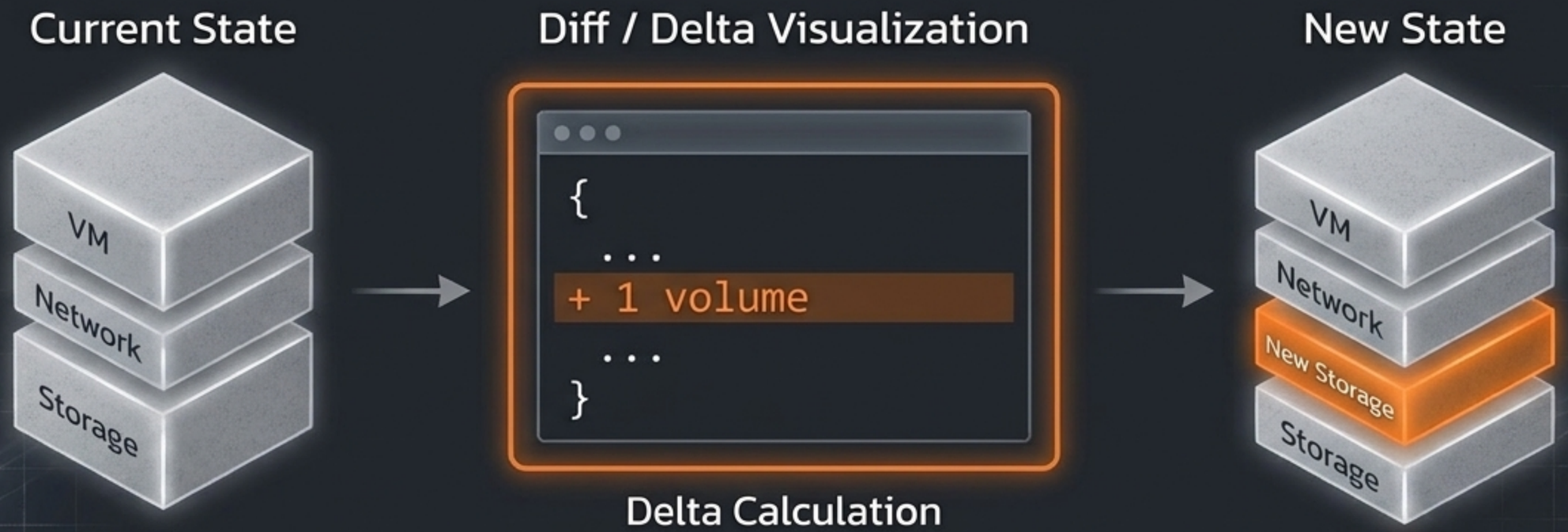
Gemini Notebook

ทำไมต้องใช้ Heat? (Manual vs. Orchestration)

	Manual Provisioning (CLI/Dashboard)	Heat Orchestration (IaC)
ความเร็วในการติดตั้ง (Speed)	ชั่วโมง/วัน	นาที/วินาที ✓
ความผิดพลาด (Error Rate)	สูง (Human Error)	ต่ำมาก (ทำงานตามโค้ด 100%) ✓
การทำซ้ำ (Repeatability)	ยากและไม่คงที่	การันตีผลลัพธ์เดิมทุกครั้ง ในทุก Environment ✓
การกู้คืนระบบ (Disaster Recovery)	ต้องสร้างใหม่ทีละส่วน	Deploy ใหม่ทั้งหมด ด้วยคำสั่งเดียว ✓

Gemini Notebook

การจัดการสถานะและการอัปเดตแบบไร้รอยต่อ (Stack Update)



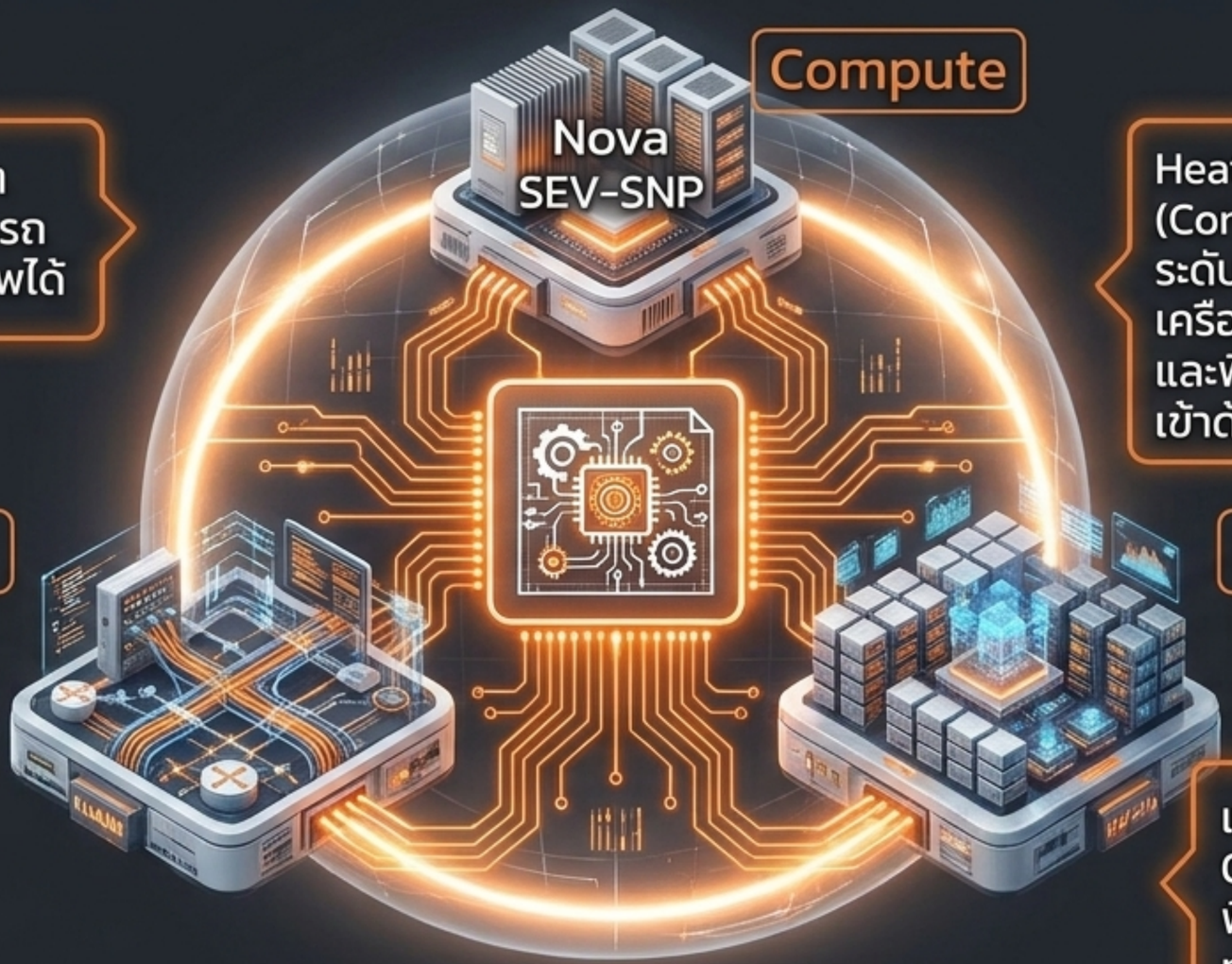
- Heat จดจำ “State” หรือสถานะปัจจุบันของระบบไว้ใน Database
- เมื่อมีการสั่ง stack update ด้วย Template ใหม่, Heat จะไม่ลบระบบเดิมทิ้งทั้งหมด
- Engine จะคำนวณส่วนต่าง (Delta) และทำเฉพาะ API Call ที่จำเป็นเท่านั้น (เช่น แค่เพิ่ม Volume ใหม่ โดยไม่กระทบ VM เดิมที่กำลังรันอยู่)

Gemini Notebook

สังเคราะห์สถาปัตยกรรม 2026.2 Hibiscus ภายใต้การควบคุมของ Heat

เทคโนโลยีขั้นสูงแบบเดี่ยวๆ (Siloed features) ไม่สามารถสร้างคลาวด์ที่มีประสิทธิภาพได้

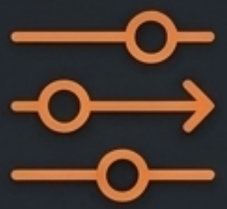
Heat ทำหน้าที่เป็น “วาทยกร” (Conductor) ที่ร้อยเรียงฟีเจอร์ระดับฮาร์ดแวร์ (TDX/SEV-SNP), เครือข่าย (BGP EVPN), และพื้นที่จัดเก็บ (Weka) เข้าด้วยกัน



แปลงศักยภาพที่ซับซ้อนของ OpenStack ให้กลายเป็นพิมพ์เขียวเดียวที่สามารถทำซ้ำและตรวจสอบได้เสมอ

Gemini Notebook

แนวทางปฏิบัติที่ดีที่สุดสำหรับการเขียน HOT



Parameterize Everything

อย่าฮาร์ดโค้ดค่าต่างๆ ใช้ Parameters เพื่อให้ Template นำไปใช้ซ้ำได้ในหลาย Environment



Use Nested Stacks

แบ่งระบบใหญ่เป็น Template ย่อยๆ (Modular) แล้วเรียกใช้กันเอง เพื่อความสะอาดของโค้ด



Embrace GitOps

เก็บไฟล์ HOT ทั้งหมดไว้ใน Version Control (Git) เพื่อทำ Audit และ CI/CD Pipeline



Leverage Outputs

ใช้ Outputs ส่งผ่านค่า (เช่น IP Address) ระหว่าง Nested Stacks เพื่อเชื่อมโยงระบบเข้าด้วยกัน

Gemini Notebook

อนาคตของคลาวด์คือการทำงานแบบ Declarative



Q&A

Gemini Notebook